
Water Pool (waterpool.*)

Escribe un Vertex Shader (VS) y un Fragment Shader (FS) que simulen los efectos de reflexión y refracción del agua en una piscina. La superficie del agua la representaremos con el modelo planeXY256.obj. El efecto tendrá dos modos controlados por el uniform int mode. Vuestra solución debe funcionar correctamente tanto si el objeto tiene transformación de modelado como si no.

Modo 0: Iluminación reflectiva [7 puntos]

Dado el plano del modelo utilizado, representaremos los movimientos del agua en nuestra piscina aplicando un displacement map. El archivo wavefunc.glsl que os da el enunciado ofrece las funciones procedurales correspondientes para calcular la altura y la derivada del desplazamiento que hay que aplicar. Tened en cuenta que las dos funciones requieren el uniform float time para funcionar.

En el VS queremos aplicar el desplazamiento, en world space, en cada vértice en el eje Y dado por la función getHeight (que se encuentra en el archivo wavefunc.glsl). Como valor de entrada le daremos la posición del vértice en el plano XZ, también en world space.

Por otra parte, queremos calcular la parte reflejada del agua en el FS. Para calcular la normal de la superficie en el FS, primero utilizaremos la función getHeightDerivative para obtener las derivadas en el eje X y Z del displacement map en ese punto. El parámetro de entrada será el mismo que en la función getHeight pero en el FS. Teniendo en cuenta que el valor de las derivadas (el resultado de la función) es un vec2 que llamaremos dxz, la normal de la superficie N corresponde a:

$$\text{vec3 } N = \text{normalize}(\text{vec3}(-dxz.x, 1.0, -dxz.y));$$

Tened en cuenta que N se encuentra en world space.

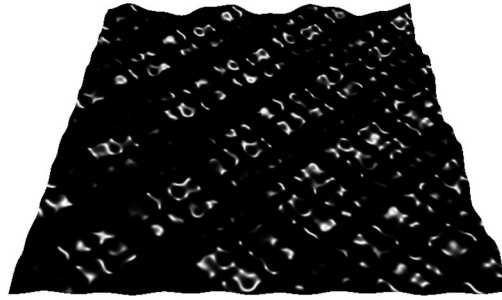
Para la iluminación de la parte reflejada supondremos solo la parte especular del modelo de Phong, teniendo en cuenta que, para simplificar:

$$KsIs = \text{vec3}(1.0)$$

Además utilizaremos un valor de shininess s de 50.0. Asumiremos una luz direccional (la dirección de la luz es siempre la misma), definida como un uniform en world space:

$$\text{uniform vec3 lightDir} = \text{vec3}(0.348, 0.87, -0.348) \text{ // en world space}$$

Calculad la iluminación en world space. Tened en cuenta que tendréis que transformar la posición de la cámara de eye space a world space (la cámara está en (0,0,0) en eye space).

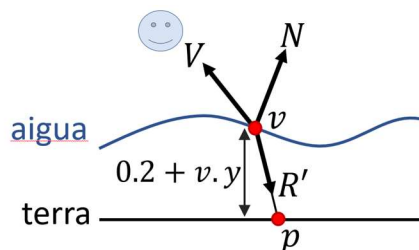


Modo 1: Refracción [3 puntos]

Para la iluminación de la parte refractada queremos simular la distorsión del fondo de la piscina teniendo en cuenta la refracción de la luz en la superficie del agua. El suelo de la piscina lo representaremos con una textura. Dado el vector V (vector normalizado del fragment al observador) y la normal del fragment N en *world space*, queremos calcular el vector refractado R' utilizando la función **refract** de GLSL:

```
vec3 refract(vec3 I, vec3 N, float eta)
```

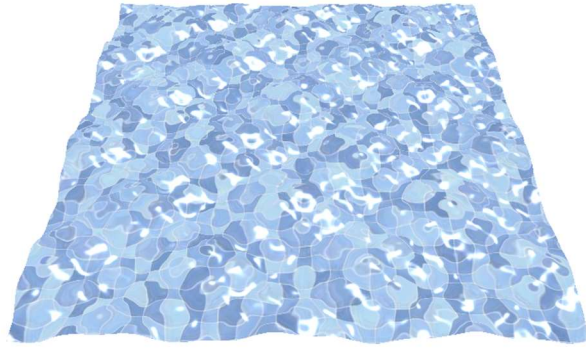
- I es el vector incidente (en nuestro caso) $-V$
- N es la normal del fragment.
- η es la proporción de los diferentes índices de refracción (en el caso agua/aire será 0.75).
- La función devuelve el vector refractado que corresponderá directamente al vector R'



Dado el vector R' en *world space* y la altura a la que se encuentra el fragment del suelo podemos calcular el punto p que corresponde al punto del suelo que el observador ve teniendo en cuenta la refracción de la luz. La altura del fragment en el suelo la calculamos teniendo en cuenta que el suelo se encuentra en el plano $y = -0.2$. El punto p se puede calcular como:

$$p = v - R' \left(\frac{0.2 + v.y}{R'.y} \right)$$

Donde el vector v corresponde a la posición del fragment en *world space*. Finalmente, el color del suelo lo calcularemos accediendo a la textura con unas coordenadas de textura iguales a la posición del punto p en el plano XZ. El color final será el color del suelo más el color calculado en el modo anterior (la componente especular de la luz reflejada):



Identificadores obligatorios:

```
uniform int mode = 0;  
uniform vec3 lightDir = vec3(0.348, 0.87, -0.348);  
uniform float time;  
uniform sampler2D poolFloor;
```

Sandman (sandman.*)

Escribe un Vertex Shader (VS), un Geometry Shader (GS) y un Fragment Shader (FS) que implementen un efecto de desintegración y animación de un modelo basado en partículas (granos de arena). El objetivo es simular que el modelo está compuesto totalmente por granos de arena que, por acción del tiempo, caen hacia el suelo y se dispersan horizontalmente. El vídeo sandman.mkv contiene el resultado final esperado para el modelo man.obj.

El VS debe encargarse exclusivamente de recibir la posición del vértice y su normal en object space, y pasarlas directamente al GS sin aplicar ninguna transformación [1 punto].

El GS recibirá triángulos como entrada y generará un único punto por cada triángulo, que representará un grano de arena independiente. Por tanto, necesitaréis esta definición en el GS:

```
layout(triangles) in;  
layout(points, max_vertices = 1) out;
```

Para cada triángulo, el GS deberá realizar los siguientes cálculos:

- **Posición y normal iniciales [1.5 puntos]:** Calculad el baricentro centre del triángulo como la media de sus tres vértices de entrada en object space. De la misma manera, calculad la normal normal media del triángulo. Normalizad la normal.
- **Ruido [0.5 puntos]:** Para que cada grano de arena actúe de manera asíncrona, implementad una función de ruido que dependa de centre y normal:
 - `noise = fract(sin(dot(center,normal)) * 100)`
 - noise debe pasarse al FS.
- **Escala y tiempo de la animación [0.5 puntos]:** La animación estará controlada por el uniform float time. Definid un factor de tiempo local t diferente para cada grano utilizando noise y un uniform float speed para variar la velocidad con la fórmula:
 - `t = fract(time * 0.2 * speed + noise)`
 - A partir de t, calculad el factor de animación factor usando la función smoothstep para que la caída esté un poco suavizada:
`factor = smoothstep(0.1, 0.9, t)`
- **Animación de caída [0.5 puntos]:** El grano debe caer en el eje Y desde su posición original centre hasta el nivel del suelo de la caja envolvente, determinado por boundingBoxMin.y. El desplazamiento dependerá linealmente de factor (si factor es 0 no se habrá movido, mientras que si es 1, habrá llegado al suelo).
- **Animación de dispersión [1.5 puntos]:** El grano también se desplazará horizontalmente en el plano XZ siguiendo la dirección normalizada de la proyección horizontal de su normal normal. La distancia máxima de dispersión

será un 5% de la diagonal de la caja envolvente. Este desplazamiento horizontal también se aplicará de manera progresiva y lineal según factor.

- **Transformación final [0.5 puntos]:** Finalmente, transformad el punto resultante a eye space para pasarlo al FS, y asignad el valor pertinente a `gl_Position`.

El FS recibirá la posición final del punto en eye space y el valor de ruido noise, y se encargará de dibujar e iluminar cada grano de arena.

- **Simulación de esfera [2 puntos]:** Por defecto, los puntos generados en OpenGL se visualizan como cuadrados bidimensionales. Debéis simular que cada grano es una pequeña esfera tridimensional utilizando la variable interna de GLSL llamada `gl_PointCoord`.
 - `gl_PointCoord` nos da las coordenadas 2D locales del fragment dentro del cuadrado que pinta el punto, variando de (0,0) a (1,1). Cambiad este espacio de coordenadas de manera que el centro del punto sea el origen (0,0) y que estas nuevas coordenadas `vec2 uv` vayan de (-0.5,-0.5) a (0.5,0.5).
 - Calculad la distancia `dist` del fragment respecto al centro del cuadrado. Si la distancia es superior a 0.5, el fragment cae fuera del círculo y debe descartarse.
 - Para los fragmentos que sí se encuentran dentro del círculo, calculad una normal 3D `N`. Utilizando la ecuación de la esfera de radio 0.5, despejad la componente `z` como `float z = sqrt(0.25 - dist*dist)`. `N`, en eye space, vendrá definida por: `N = normalize(vec3(uv.x, uv.y, z))`.
- **Color e iluminación [2 puntos]:**
 - El color base del grano `sandColor` será una interpolación lineal de dos colores de arena constantes: (0.80, 0.60, 0.40) y (0.95, 0.85, 0.70). Utilizad el valor de ruido recibido del GS como parámetro de interpolación lineal para que cada grano adopte una tonalidad ligeramente diferente.
 - Aplicad un modelo de iluminación básico, multiplicando el color anterior por la componente `Z` de la normal `N` calculada antes.

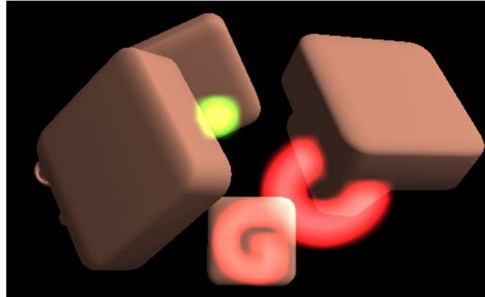
Identificadores obligatorios:

sandman.vert, sandman.geom, sandman.frag

```
uniform mat4 modelViewProjectionMatrix;  
uniform mat4 modelViewMatrix;  
uniform float time;  
uniform float speed = 1.0;  
uniform vec3 boundingBoxMin;  
uniform vec3 boundingBoxMax;
```

Bloom (bloom.*)

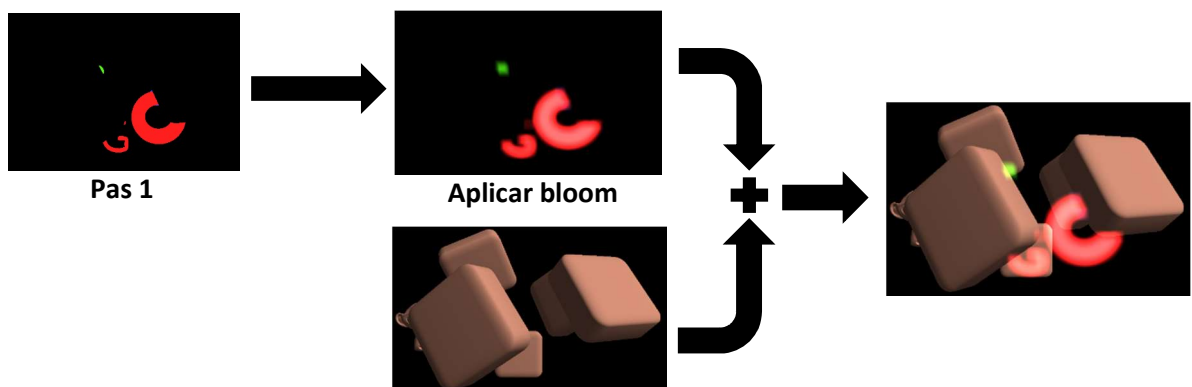
Os proporcionamos un render plugin que implementa parcialmente un efecto de bloom en algunos objetos de la escena. Este efecto se utiliza para crear la ilusión de que un objeto está irradiando luz y es muy brillante. El efecto consiste en difuminar el color solo de los objetos que queremos simular que están irradiando luz. La escena final deberá verse así:



El código que os proporcionamos ya se encarga de cargar todos los modelos, shaders y texturas necesarias. Además os damos dos funciones que renderizan los dos tipos de objetos que tenemos en la escena: `drawNormalObjects()` renderiza los objetos a los que no queremos aplicar el efecto de bloom, y la función `drawBloomObjects()` renderiza los objetos a los que queremos aplicar el efecto.

Tendréis que completar las secciones de código que están indicadas con un comentario “TODO:”. Queremos que el código haga lo siguiente (hay partes que ya están hechas, leed la página siguiente):

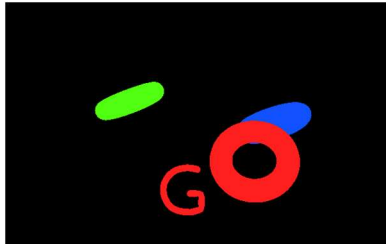
- **Paso 1:** En un primer paso, queremos renderizar solo las partes de los objetos donde aplicaremos el efecto de bloom, teniendo en cuenta las oclusiones de la escena completa (es decir, renderizando `drawNormalObjects()` en el depth buffer, pero evitando que los objetos se vean en el color buffer final). Finalmente, guardamos la escena en una textura para aplicar posteriormente el efecto de bloom.
- **Paso 2:** En este paso, primero queremos renderizar la escena normal (`drawNormalObjects()`). Seguidamente, activando alpha blending y sin tener en cuenta el depth buffer, queremos pintar la textura renderizada en el paso 1 en un quad que ocupe todo el viewport utilizando el shader de bloom (variable `programBloom`) para que pinte la textura aplicando el característico difuminado del bloom.



Seguidamente, tenéis una explicación de lo que falta por completar:

Paso 1

Os damos un código que renderiza los objetos a los que aplicar bloom, pero no tiene en cuenta las oclusiones de los otros objetos (los que se renderizan con la función `drawNormalObjects()`). En el código veréis que hay una línea comentada al final del paso 1 que dice `return true`; podéis descomentar la línea para ver el resultado parcial del resultado 1 (recordad comentarla de nuevo).



Sin tener en cuenta los otros objetos



Teniendo en cuenta los otros objetos

Paso 2

En el paso dos se pintan los objetos de la escena que no necesitan bloom, pero faltan los otros. Debéis implementar el código para pintar la textura guardada en el paso 1 con el shader de bloom que ya os damos (variable `programBloom`). La textura la queréis pintar de tal forma que se vea todo el viewport de la escena. En el código que os damos tenéis la función `drawRect` que dibuja un cuadrado con coordenadas de $(-1, -1, 0)$ a $(1, 1, 0)$. El cuadrado deberéis pintarlo de tal forma que ocupe todo el viewport, sin tener en cuenta el depth test. Pensad que también deberéis pasar los uniforms necesarios al shader de bloom. Los uniforms son los siguientes:

- *modelViewProjectionMatrix*: Recordad que no queremos utilizar la matriz de la cámara en este caso, ya que queremos que las coordenadas en object space sean las mismas que en clip space.
- *colorMap*: En este caso queremos que el shader tenga acceso a la textura que hemos rellenado en el paso 1.
- *SIZE*: El shader también necesita las dimensiones del viewport en píxeles para hacer el cálculo correctamente (el propio plugin fuerza un tamaño fijo).

IMPORTANTE: Debéis entregar el ejercicio como un zip o tar con la carpeta bloom con vuestra solución completa (debemos poder incluir esta carpeta en la carpeta de plugins del viewer y poder ejecutarlo con GLarenaPL). No incluir la carpeta build ni el archivo Makefile. (podéis ejecutar `make distclean` dentro de la carpeta del plugin para eliminarlas).